p3_1

Exported 2/21/2026, 10:11:20 AM

3.1

The document was converted with **Code-Format**: https://code-format.com/

In [2]:

```
import os, sys, numpy as np, matplotlib.pyplot as plt
sys.path.append('../hw1')
import heat_rod as hr
from mcmc import *
from scipy.io import loadmat

data_name ="HW06_Problem3.mat"
samples, q_map, posterior_values, acceptance_rate =
run_metropolis_for_heat_equation(data_name = data_name, M = 10000)

# Save results
np.savez('problem3.1_results.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)

print(f"\nResults saved to 'problem3.1_results.npz'")
```

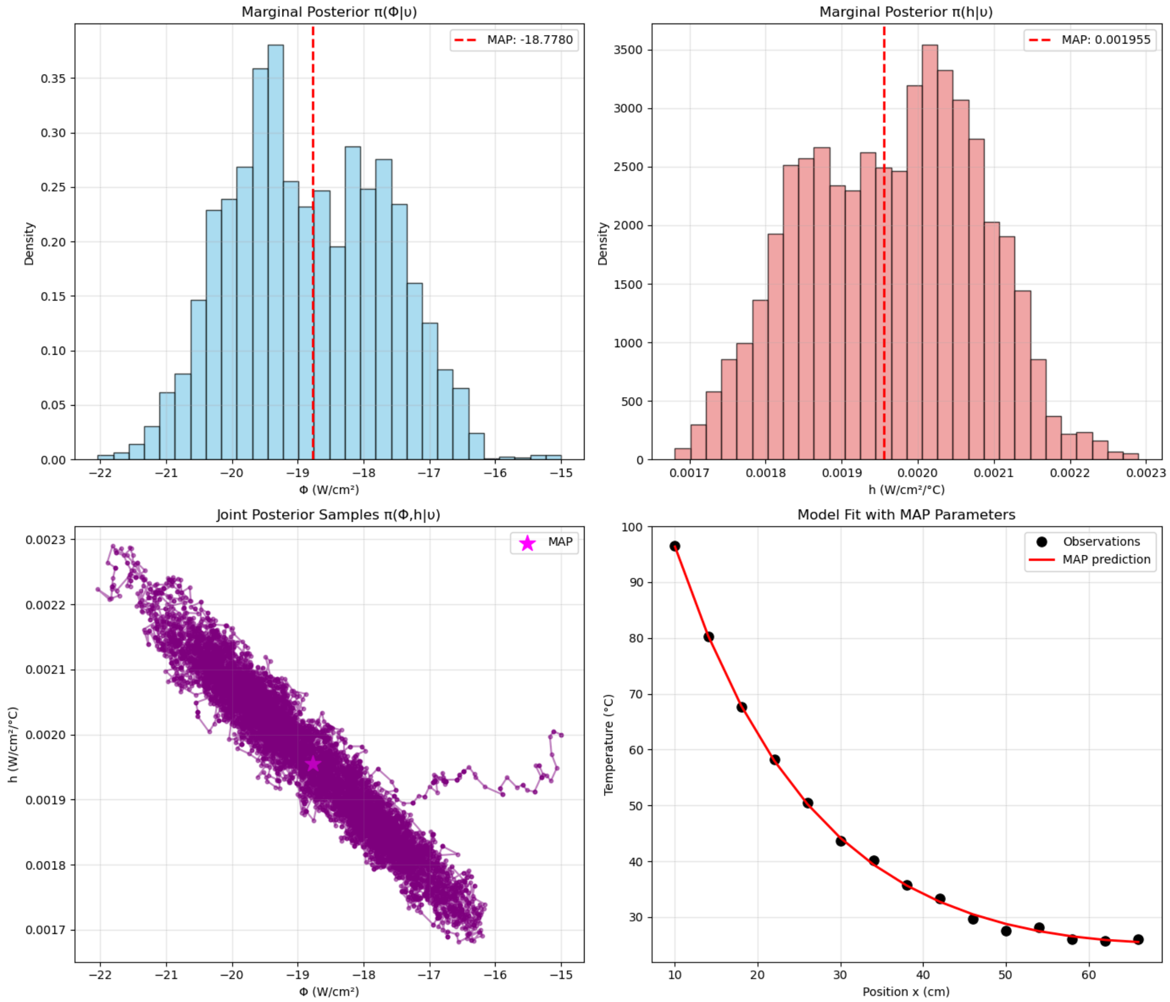
The document was converted with **Code-Format**: https://code-format.com/

```
Shape of observations: (15,)
Initial test - r(q0, q0) = 1.000000

Running Metropolis algorithm for 10000 iterations...
Proposal distribution: N(q_prev, D) with D = diag([0.01, 2e-10])
Running Metropolis-Hastings MCMC...
  Completed 200/10000 iterations, acceptance rate: 0.729
  Completed 400/10000 iterations, acceptance rate: 0.772
  Completed 600/10000 iterations, acceptance rate: 0.798
  Completed 800/10000 iterations, acceptance rate: 0.814
  Completed 1000/10000 iterations, acceptance rate: 0.818
  Completed 1200/10000 iterations, acceptance rate: 0.826
  Completed 1400/10000 iterations, acceptance rate: 0.832
  Completed 1600/10000 iterations, acceptance rate: 0.836
  Completed 1800/10000 iterations, acceptance rate: 0.840
  Completed 2000/10000 iterations, acceptance rate: 0.835
  Completed 2200/10000 iterations, acceptance rate: 0.834
  Completed 2400/10000 iterations, acceptance rate: 0.835
  Completed 2600/10000 iterations, acceptance rate: 0.838
  Completed 2800/10000 iterations, acceptance rate: 0.835
  Completed 3000/10000 iterations, acceptance rate: 0.835
  Completed 3200/10000 iterations, acceptance rate: 0.834
  Completed 3400/10000 iterations, acceptance rate: 0.833
  Completed 3600/10000 iterations, acceptance rate: 0.832
  Completed 3800/10000 iterations, acceptance rate: 0.828
  Completed 4000/10000 iterations, acceptance rate: 0.827
  Completed 4200/10000 iterations, acceptance rate: 0.826
  Completed 4400/10000 iterations, acceptance rate: 0.826
  Completed 4600/10000 iterations, acceptance rate: 0.826
  Completed 4800/10000 iterations, acceptance rate: 0.825
  Completed 5000/10000 iterations, acceptance rate: 0.826
  Completed 5200/10000 iterations, acceptance rate: 0.827
  Completed 5400/10000 iterations, acceptance rate: 0.827
  Completed 5600/10000 iterations, acceptance rate: 0.826
  Completed 5800/10000 iterations, acceptance rate: 0.826
  Completed 6000/10000 iterations, acceptance rate: 0.826
  Completed 6200/10000 iterations, acceptance rate: 0.824
  Completed 6400/10000 iterations, acceptance rate: 0.825
  Completed 6600/10000 iterations, acceptance rate: 0.826
  Completed 6800/10000 iterations, acceptance rate: 0.826
  Completed 7000/10000 iterations, acceptance rate: 0.826
  Completed 7200/10000 iterations, acceptance rate: 0.825
  Completed 7400/10000 iterations, acceptance rate: 0.825
  Completed 7600/10000 iterations, acceptance rate: 0.825
  Completed 7800/10000 iterations, acceptance rate: 0.824
  Completed 8000/10000 iterations, acceptance rate: 0.823
```

The document was converted with **Code-Format**: https://code-format.com/

```
  Completed 8200/10000 iterations, acceptance rate: 0.822
  Completed 8400/10000 iterations, acceptance rate: 0.821
  Completed 8600/10000 iterations, acceptance rate: 0.821
  Completed 8800/10000 iterations, acceptance rate: 0.821
  Completed 9000/10000 iterations, acceptance rate: 0.822
  Completed 9200/10000 iterations, acceptance rate: 0.822
  Completed 9400/10000 iterations, acceptance rate: 0.821
  Completed 9600/10000 iterations, acceptance rate: 0.821
  Completed 9800/10000 iterations, acceptance rate: 0.820
  Completed 10000/10000 iterations, acceptance rate: 0.820

Final acceptance rate: 0.820

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -18.778007 W/cm²
  h = 0.001955 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 1.61e+00

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -18.830713 W/cm²
  Std  = 1.159654 W/cm²
  95% CI = [-20.887713, -16.661970]

h:
  Mean = 1.964490e-03 W/cm²/°C
  Std  = 1.145543e-04 W/cm²/°C
  95% CI = [1.748568e-03, 2.164453e-03]
```

The document was converted with **Code-Format**: https://code-format.com/




```
Results saved to 'problem3.1_results.npz'
```

$^{22}q_{MAP} = [-18.72W/cm^2; h = 0.00195W/cm^2/°C]$ after removing 300 burn-in samples.

In [3]:

```
burnin = 300
q_map_after_burnin = samples[np.argmax(posterior_values[burnin:])]
print(f"MAP point after removing {burnin} burn-in points",
q_map_after_burnin)
print(f"MAP point wihout removing {burnin} burn-in points", q_map)
```

```
MAP point after removing 300 burn-in points [-1.87244917e+01
1.95073235e-03]
MAP point wihout removing 300 burn-in points [-1.87780074e+01
1.95508105e-03]
```

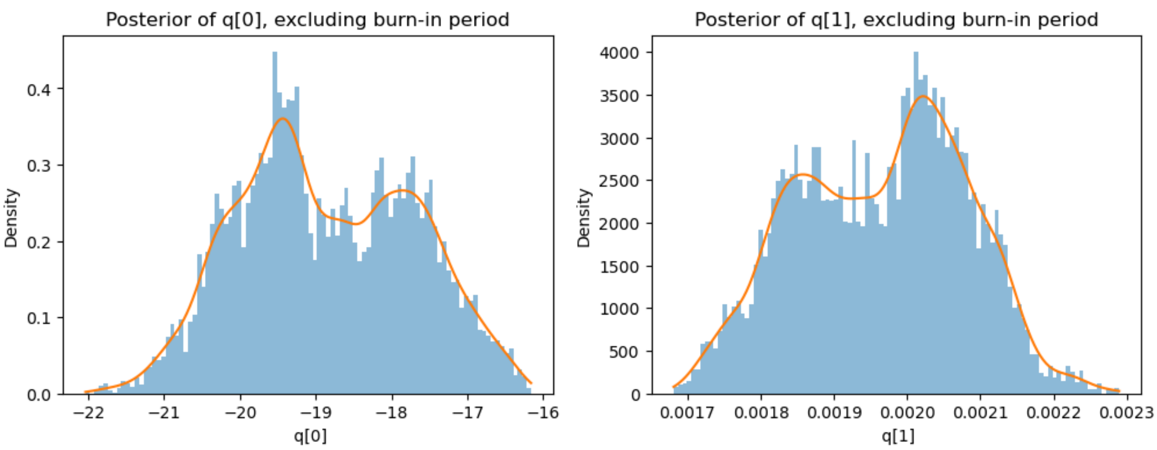
The document was converted with **Code-Format**: https://code-format.com/

Histograms of Samples

KDE of PDF

In [5]:

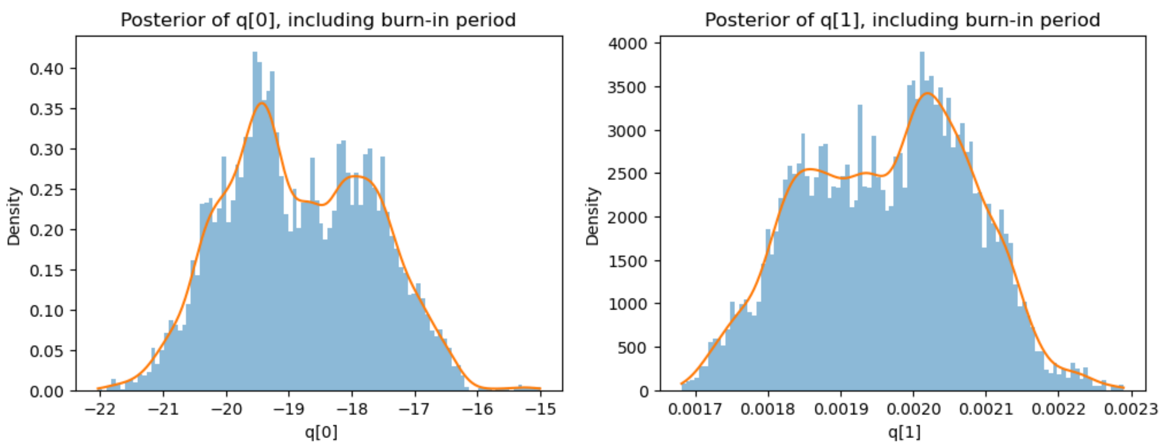
```
from scipy.stats import gaussian_kde
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for i in range(2):
    s = samples[burnin:, i] ####TODO: BURNIN INCLUDED?

    # Histogram
    axes[i].hist(s, bins=100, density=True, alpha=0.5)

    # KDE smooth curve
    kde = gaussian_kde(s)
    x = np.linspace(min(s), max(s), 300)
    axes[i].plot(x, kde(x))

    axes[i].set_title(f"Posterior of q[{i}], excluding burn-in
period")
    axes[i].set_xlabel(f" q[{i}]")
    axes[i].set_ylabel("Density")
```



The document was converted with **Code-Format**: https://code-format.com/

In [6]:

```
from scipy.stats import gaussian_kde
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for i in range(2):
    s = samples[:, i] ####TODO: BURNIN INCLUDED?

    # Histogram
    axes[i].hist(s, bins=100, density=True, alpha=0.5)

    # KDE smooth curve
    kde = gaussian_kde(s)
    x = np.linspace(min(s), max(s), 300)
    axes[i].plot(x, kde(x))

    axes[i].set_title(f"Posterior of q[{i}], including burn-in 
period")
    axes[i].set_xlabel(f" q[{i}]")
    axes[i].set_ylabel("Density")
```



Note: q[0] is ϕ, while q[1] is h.